5. Random Numbers

5.1. Motivation

Random numbers are one of the central tools of computational physics. They are used whenever we want to represent uncertainty, sample a probability distribution, model fluctuations, or approximate high-dimensional integrals. Important examples include Monte Carlo integration, random walks and diffusion, Langevin dynamics, radiative transfer, particle transport, statistical mechanics, Bayesian inference, and uncertainty propagation.

At first sight, the idea of random numbers in a computer seems paradoxical. A computer is a deterministic machine: if we run the same program twice with the same input, we expect exactly the same result. Most random numbers used in scientific computing are therefore not truly random. They are pseudo-random numbers: deterministic sequences that are designed to behave statistically like random numbers.

This apparent weakness is also a strength. Since a pseudo-random sequence is fully determined by its initial seed, a stochastic simulation can be repeated exactly. Reproducibility is therefore one of the most important practical advantages of pseudo-random number generators.

5.2. What should a random sequence look like?

A sequence of random numbers should satisfy several statistical properties.

- **Uniformity:** values should occur with the correct probability.
- **Independence:** knowing one value should not help to predict the next one.
- **Long period:** the sequence should not repeat on relevant time scales.
- **Lack of visible structure:** points formed from consecutive values should not lie on obvious lines, planes, or other low-dimensional patterns.
- **Reproducibility:** the same seed should produce the same sequence.

A useful distinction is therefore not simply between “random” and “not random”, but between different levels of randomness needed for different applications. A generator that is good enough for a classroom Monte Carlo integral may be unacceptable for cryptography. Conversely, a cryptographically secure generator may be unnecessarily slow for a large numerical simulation.

5.3. Uniform random numbers

Most random number generators first produce numbers that are approximately uniformly distributed on the interval $[0, 1)$. The corresponding probability density is

$$p(x) = \begin{cases} 1, & 0 \leq x < 1, \\ 0, & \text{otherwise.} \end{cases} \tag{5.1}$$

The expectation value is

$$\langle x \rangle = \int_0^1 x \, dx = \frac{1}{2}, \tag{5.2}$$

and the second moment is

$$\langle x^2 \rangle = \int_0^1 x^2 \, dx = \frac{1}{3}. \tag{5.3}$$

The variance is therefore

$$\mathrm{Var}(x) = \langle x^2 \rangle - \langle x \rangle^2 = \frac{1}{3} - \frac{1}{4} = \frac{1}{12}. \tag{5.4}$$

These simple results are useful reference values for testing a generator.

5.4. Pseudo-random number generators

5.4.1. Linear congruential generators

One of the simplest pseudo-random number generators is the linear congruential generator (LCG), defined by

$$X_{n+1} = (aX_n + c) \bmod m, \tag{5.5}$$

where m is the modulus, a the multiplier, c the increment, and X_0 the seed. The normalized output is usually

$$u_n = \frac{X_n}{m}, \tag{5.6}$$

so that $u_n \in [0, 1)$.

The LCG is useful pedagogically because it makes the deterministic nature of pseudo-random numbers explicit. Since there are only m possible states, the sequence must eventually repeat. With a poor choice of parameters, the period can be short and correlations can be strong. With a better choice, the sequence may appear random for many simple applications, but LCGs are generally not recommended for demanding modern simulations.

Figure 5.1 illustrates this point. A poor LCG can produce points with a visible lattice structure when consecutive values are plotted against each other.

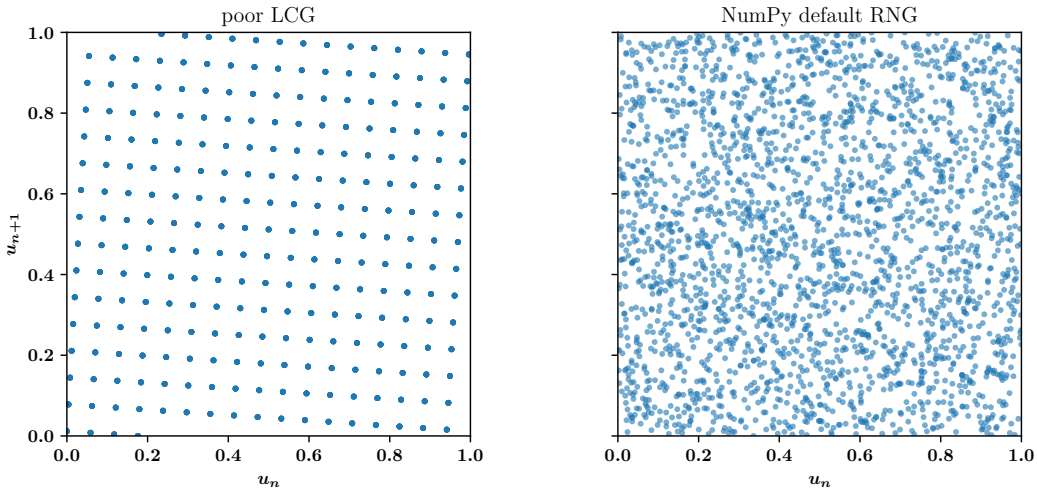

Figure 5.1.: Consecutive random numbers plotted as pairs (u_n, u_{n+1}). A poor generator can show obvious lattice structure, whereas a better generator fills the unit square more uniformly.

5.4.2. Modern generators

Modern simulations usually rely on better generators than simple LCGs. Common examples are the Mersenne Twister, XORSHIFT-type generators, PCG generators, and counter-based generators. Their details differ, but the goal is always the same: produce fast, reproducible, statistically reliable pseudo-random sequences with long periods and weak correlations.

For parallel simulations, one must also ensure that different processes or threads do not accidentally use identical or strongly overlapping random sequences. A common mistake is to initialize every worker with the same seed. Good parallel random number generation requires either independent streams, carefully constructed seed sequences, or counter-based generators.

5.5. Statistical tests

No finite test can prove that a sequence is truly random. Tests can only detect certain kinds of non-randomness. Nevertheless, simple tests are very useful diagnostics.

5.5.1. Expected statistical fluctuations in histogram bins

For a uniform random number generator, the number of samples in equal-width bins should be approximately equal. However, even a perfect random number generator does not produce exactly identical bin populations. Instead, the number of samples fluctuates statistically.

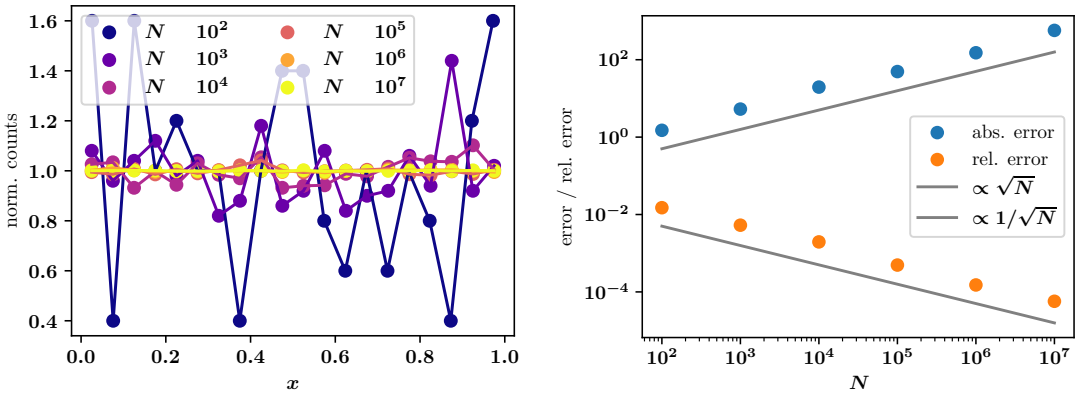

Figure 5.2.: Histogram convergence for uniformly distributed random numbers. As the sample size increases, relative fluctuations decrease, but the histogram is never perfectly flat for finite N.

Assume that we generate N independent random numbers and distribute them into B equal-width bins. For a perfectly uniform distribution, the probability for a random number to fall into a particular bin is

$$p = \frac{1}{B} \tag{5.7}$$

The number of samples n inside one specific bin therefore follows a binomial distribution:

$$P(n) = \binom{N}{n} p^n (1-p)^{N-n} \tag{5.8}$$

For a binomial distribution, the expectation value and variance are

$$\mu = Np \quad \text{and} \quad \mathrm{Var}(n) = Np(1-p). \tag{5.9}$$

Using $p = 1/B$, the expected number of samples per bin becomes

$$\mu = \frac{N}{B} \tag{5.10}$$

and the variance is

$$\mathrm{Var}(n) = \frac{N}{B}\left(1 - \frac{1}{B}\right) \tag{5.11}$$

For a sufficiently large number of bins ($B \gg 1$), we can approximate

$$1 - \frac{1}{B} \approx 1 \tag{5.12}$$

which leads to

$$\mathrm{Var}(n) \approx \frac{N}{B} = \mu \tag{5.13}$$

Therefore, the standard deviation of the fluctuations is approximately

$$\sigma = \sqrt{\mathrm{Var}(n)} \approx \sqrt{\mu}. \tag{5.14}$$

This behavior is characteristic of Poisson statistics, where the variance is equal to the mean. An important consequence is that the absolute fluctuations increase as

$$\sigma \propto \sqrt{N} \tag{5.15}$$

while the mean grows linearly with N. The relative fluctuations therefore decrease as

$$\frac{\sigma}{\mu} \approx \frac{1}{\sqrt{\mu}} \tag{5.16}$$

This explains why histograms become smoother when more random numbers are generated, see Fig. 5.2. The same statistical scaling also appears in Monte Carlo integration and leads to the characteristic convergence behavior

$$\mathrm{error}_{\mathrm{MC}} \propto N^{-1/2} \tag{5.17}$$

which reflects the fundamental Poisson noise limit of random sampling methods.

5.5.2. Autocorrelation

A generator should not produce strongly correlated consecutive values. A simple diagnostic is the autocorrelation at lag k,

$$C(k) = \frac{\sum_{i=1}^{N-k}(x_i - \bar{x})(x_{i+k} - \bar{x})}{\sum_{i=1}^{N}(x_i - \bar{x})^2}. \tag{5.18}$$

For a good independent sequence, $C(k)$ should be close to zero for $k > 0$, up to statistical fluctuations.

5.6. Non-uniform random number distributions

Uniform random numbers are the starting point, but physical applications often require other distributions: exponential waiting times, Gaussian velocities, power-law particle energies, or arbitrary probability densities. We therefore need methods for transforming uniform samples into samples from a desired distribution.

5.6.1. Probability densities and cumulative distribution functions

Let $p(x)$ be a normalized probability density,

$$\int_{-\infty}^{\infty} p(x)\,dx = 1. \tag{5.19}$$

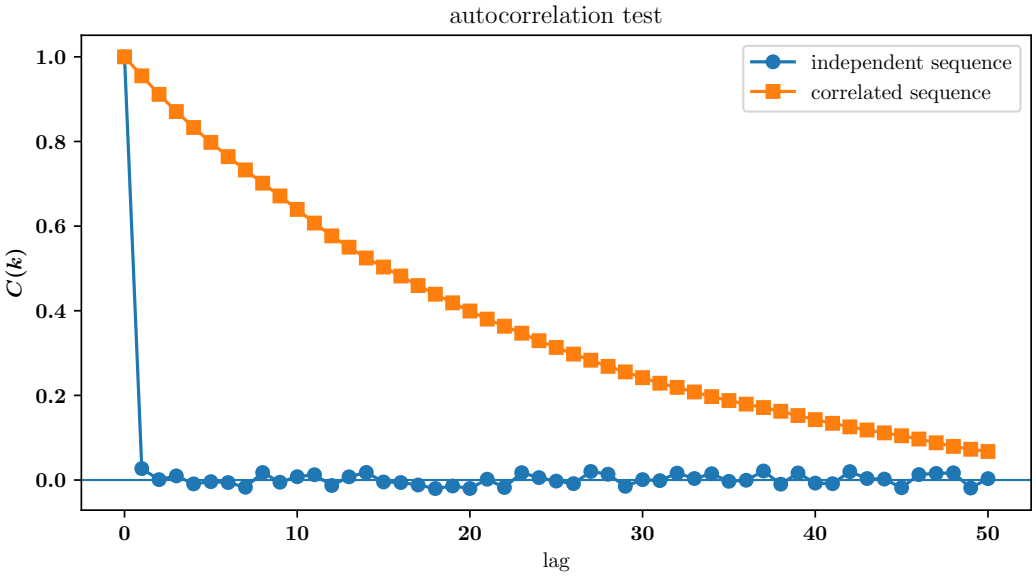

Figure 5.3.: Autocorrelation for an approximately independent random sequence and for a deliberately correlated sequence. Correlations reduce the effective number of independent samples.

The cumulative distribution function is

$$F(x) = \int_{-\infty}^{x} p(x')\,dx'. \tag{5.20}$$

It gives the probability that a random variable X is smaller than or equal to x,

$$F(x) = P(X \leq x). \tag{5.21}$$

Because $p(x)$ is normalized, the cumulative distribution satisfies

$$0 \leq F(x) \leq 1 \tag{5.22}$$

and increases monotonically from 0 to 1.

example: A simple, often used distribution are uniformly distributed random numbers between 0 and 1, $\mathcal{U}(0,1)$. The probability density function is a constant $p(x) = p_{\mathcal{U}}$. Normalisation of the probability requires

$$1 \equiv \int_0^1 p_{\mathcal{U}}\,\mathrm{d}x = p_{\mathcal{U}}. \tag{5.23}$$

The corresponding CDF is a linear function

$$F(x) = p_{\mathcal{U}}\,x = x. \tag{5.24}$$

5.6.2. Direct inversion sampling / inverse transform sampling

We need random numbers that follow a desired probability distribution $p(x)$. However, most random number generators only produce uniformly distributed random numbers in the interval $[0, 1]$. Therefore, we need methods to transform these uniform random numbers into samples following other distributions. The direct inversion method is based on the cumulative distribution function (CDF). The key idea is the following:

- We first generate a uniformly distributed random number $u \sim \mathcal{U}(0, 1)$.
- We then compute

$$x = F^{-1}(u) \tag{5.25}$$

  where F^{-1} is the inverse cumulative distribution function.

The resulting variable x is then distributed according to the desired probability density $p(x)$.

Why does this work?

The reason can be understood geometrically. A uniform random number generator produces numbers that are evenly distributed between 0 and 1. Therefore, the probability that u lies inside an interval $[u_1, u_2]$ is simply

$$P(u_1 < u < u_2) = u_2 - u_1 \tag{5.26}$$

Now consider the transformation

$$u = F(x) \tag{5.27}$$

Since $F(x)$ represents the accumulated probability up to x, equal intervals in u-space correspond to equal probabilities in x-space.

For example:

- If $p(x)$ is large in some region, then $F(x)$ increases rapidly. A small interval in x therefore corresponds to a relatively large interval in u.
- If $p(x)$ is small, then $F(x)$ grows slowly, and a larger interval in x is needed to accumulate the same probability.

As a result, the transformation automatically produces more samples in regions where the probability density $p(x)$ is large, which is illustrated in Fig. 5.4.

Formal derivation

We can derive the result mathematically using probability conservation. The probability to find a sample inside an interval must remain unchanged under the transformation:

$$p(x)\,dx = p(u)\,du \tag{5.28}$$

Since u is uniformly distributed on $[0, 1]$,

$$p(u) = 1 \tag{5.29}$$

Using

$$u = F(x) \tag{5.30}$$

we obtain

$$du = \frac{dF}{dx}\,dx \tag{5.31}$$

Because the derivative of the cumulative distribution is the probability density,

$$\frac{dF}{dx} = p(x) \tag{5.32}$$

we finally obtain

$$du = p(x)\,dx \tag{5.33}$$

which is exactly the required probability distribution. Therefore, the transformed variable

$$x = F^{-1}(u) \tag{5.34}$$

is distributed according to $p(x)$.

Example: Exponential distribution

As an example, consider the exponential distribution

$$p(x) = \lambda \exp(-\lambda x), \qquad x \geq 0. \tag{5.35}$$

The cumulative distribution is

$$F(x) = \int_0^x \lambda \exp\bigl(-\lambda x'\bigr)\,dx' = \bigl[-\exp(-\lambda x)\bigr]_0^x = 1 - \exp(-\lambda x). \tag{5.36}$$

Setting $F(x) = u$ gives

$$u = 1 - \exp(-\lambda x). \tag{5.37}$$

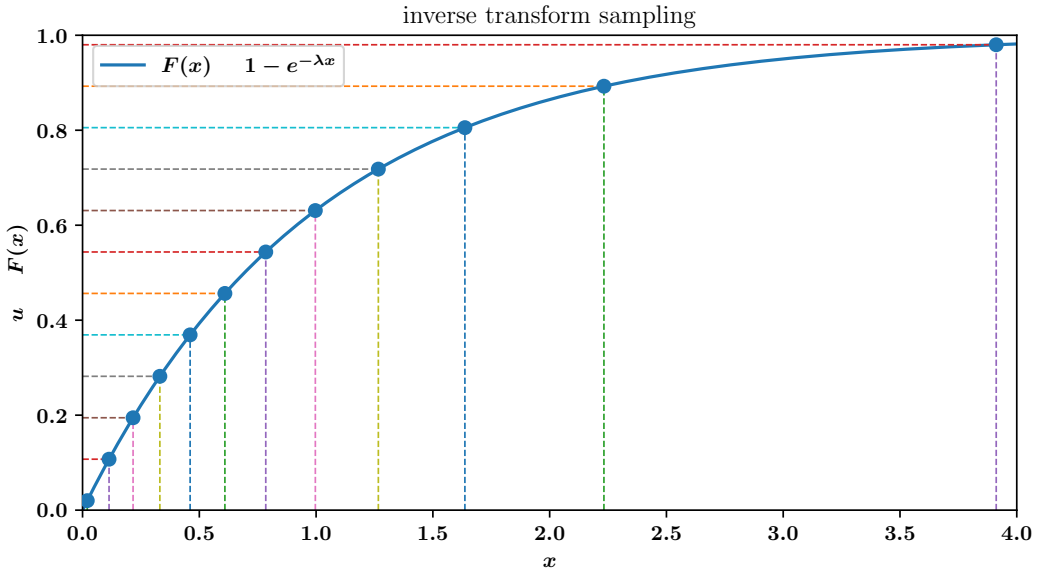

Figure 5.4.: Inverse transform sampling for an exponential distribution. Uniform samples on the vertical axis are mapped through the inverse cumulative distribution function to non-uniform samples on the horizontal axis.

Solving for x yields

$$x = -\frac{1}{\lambda}\ln(1-u). \tag{5.38}$$

Since $1-u$ is also uniformly distributed on $[0,1)$, one often writes

$$x = -\frac{1}{\lambda}\ln u. \tag{5.39}$$

This distribution appears in radioactive decay, collision distances, photon absorption, Poisson processes, and many other waiting-time problems.

5.6.3. Box–Muller transform

The Box–Muller transform is a classical method for generating Gaussian random numbers from uniformly distributed random numbers. Suppose we want random numbers distributed according to the standard normal distribution

$$p(x) = \frac{1}{\sqrt{2\pi}}e^{-x^2/2} \tag{5.40}$$

with mean zero and variance one. The Box–Muller method starts from two independent uniform random numbers

$$u_1, u_2 \sim \mathcal{U}(0,1) \tag{5.41}$$

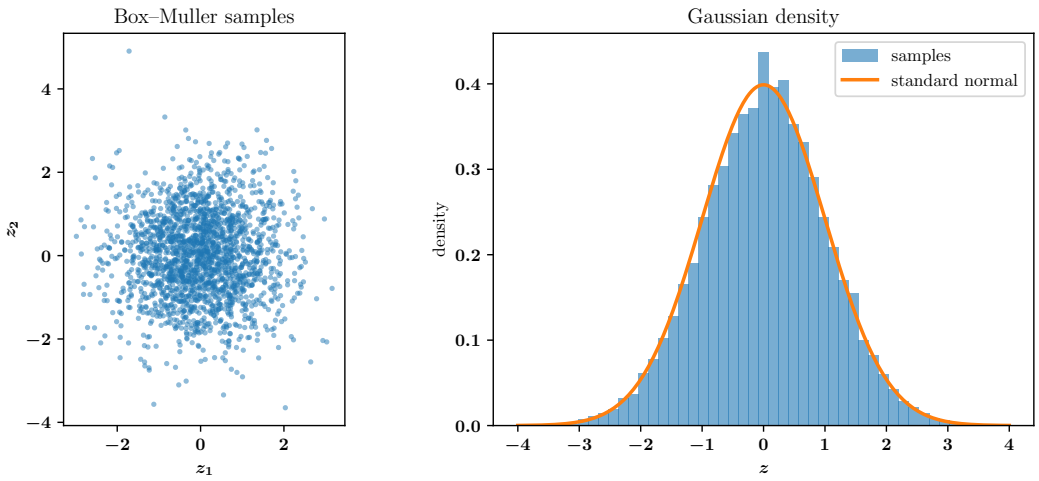

Figure 5.5.: Gaussian random numbers generated with the Box–Muller transform. The left panel shows samples in the (z_1, z_2) plane; the right panel shows the one-dimensional histogram compared to the standard normal density.

and transforms them into two Gaussian random numbers

$$z_1 = \sqrt{-2 \ln u_1} \cos(2\pi u_2) \tag{5.42}$$

$$z_2 = \sqrt{-2 \ln u_1} \sin(2\pi u_2) \tag{5.43}$$

Both z_1 and z_2 are independent normally distributed random numbers with mean zero and variance one, see Fig. 5.5. The notation for Gaussian random numbers (normally distributed numbers) is $\mathcal{N}(\text{mean}, \text{std-dev})$, which in many cases is $\mathcal{N}(0, 1)$.

Basic idea

The derivation becomes particularly elegant when considering the two-dimensional Gaussian distribution.

Suppose two random variables z_1 and z_2 are independent standard Gaussians. Their joint probability density is

$$p(z_1, z_2) = \frac{1}{2\pi} e^{-(z_1^2 + z_2^2)/2} \tag{5.44}$$

This distribution is rotationally symmetric because it only depends on

$$r^2 = z_1^2 + z_2^2 \tag{5.45}$$

Therefore, it is natural to transform to polar coordinates:

$$z_1 = r \cos \phi \tag{5.46}$$

$$z_2 = r \sin \phi \tag{5.47}$$

The area element transforms as

$$dz_1\, dz_2 = r\, dr\, d\phi \tag{5.48}$$

and the probability density becomes

$$p(r, \phi) = \frac{1}{2\pi} e^{-r^2/2} r \tag{5.49}$$

This expression factorizes into

$$p(r, \phi) = \left(r e^{-r^2/2} \right) \left(\frac{1}{2\pi} \right), \tag{5.50}$$

which means:

- the angle ϕ is uniformly distributed between 0 and 2π,
- the radius r follows the distribution

$$p(r) = r e^{-r^2/2} \tag{5.51}$$

Generating the angle

A uniformly distributed angle is easy to generate:

$$\phi = 2\pi u_2 \tag{5.52}$$

because u_2 is uniform in $[0, 1]$.

Generating the radius

The radial distribution can be generated using direct inversion. The cumulative distribution is

$$F(r) = \int_0^r r' e^{-r'^2/2}\, dr' \tag{5.53}$$

which evaluates to

$$F(r) = 1 - e^{-r^2/2} \tag{5.54}$$

Setting

$$u_1 = F(r) \tag{5.55}$$

gives

$$u_1 = 1 - e^{-r^2/2} \tag{5.56}$$

Solving for r yields

$$r = \sqrt{-2\ln(1-u_1)} \tag{5.57}$$

Since $1-u_1$ is also uniformly distributed, this is usually written as

$$r = \sqrt{-2\ln u_1} \tag{5.58}$$

Combining radius and angle gives the final Box–Muller formulas:

$$z_1 = \sqrt{-2\ln u_1}\cos(2\pi u_2) \tag{5.59}$$

$$z_2 = \sqrt{-2\ln u_1}\sin(2\pi u_2) \tag{5.60}$$

5.6.4. Rejection sampling

Inverse transform sampling is powerful, but it requires the inverse cumulative distribution function. For many distributions this inverse is unavailable or expensive. Rejection sampling provides a more general alternative.

Assume we want to sample from a target density $p(x)$. We choose an envelope function $Mg(x)$, where $g(x)$ is a proposal distribution that can be sampled easily and M is a constant such that

$$p(x) \leq Mg(x) \tag{5.61}$$

for all relevant x. The algorithm is:

1. Draw a candidate x from $g(x)$.
2. Draw a uniform random number $u \in [0,1)$.
3. Accept the candidate if
$$u \leq \frac{p(x)}{Mg(x)}. \tag{5.62}$$
4. Otherwise reject the candidate and try again.

The accepted samples are distributed according to $p(x)$. The method is simple and general, but it becomes inefficient if the envelope is much larger than the target distribution.

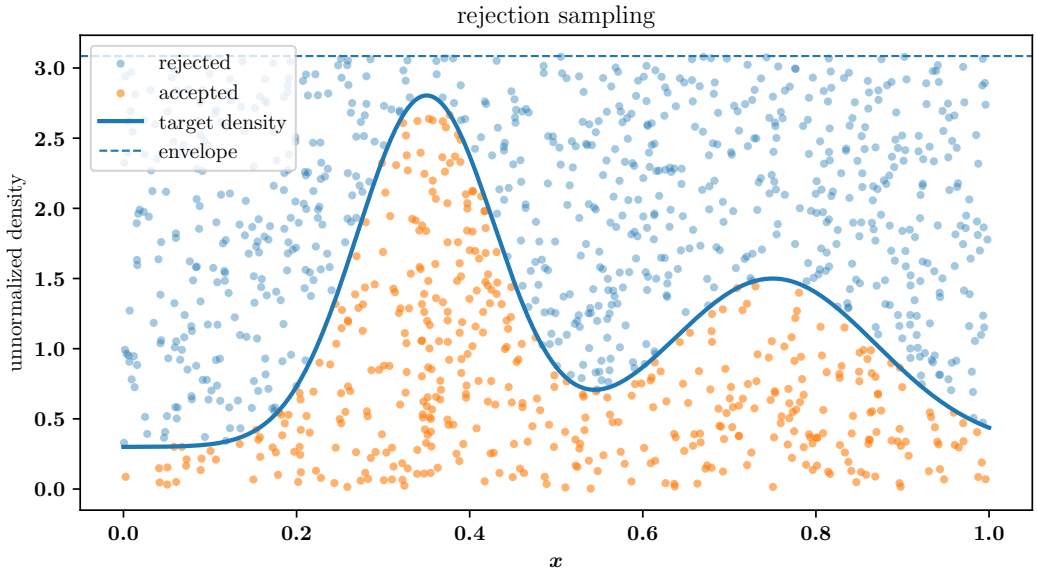

Figure 5.6.: Rejection sampling for a target distribution. Candidate points below the target curve are accepted, while points above it are rejected.

5.7. Monte Carlo integration

Monte Carlo integration is one of the most important applications of random numbers in computational physics. Let $f(x)$ be an integrable function over a domain D with volume V_D. The integral is

$$I = \int_D f(x)\,dx. \tag{5.63}$$

If we draw N independent points x_i uniformly from D, then

$$I \approx \frac{V_D}{N} \sum_{i=1}^{N} f(x_i). \tag{5.64}$$

The estimator is unbiased, because

$$\left\langle \frac{V_D}{N} \sum_{i=1}^{N} f(x_i) \right\rangle = V_D \langle f \rangle = \int_D f(x)\,dx. \tag{5.65}$$

The variance of the estimator is

$$\mathrm{Var}(I_N) = V_D^2\,\mathrm{Var}(\bar{f}) = \frac{V_D^2}{N}\mathrm{Var}(f). \tag{5.66}$$

The error therefore decreases as

$$\epsilon \propto \frac{1}{\sqrt{N}}. \tag{5.67}$$

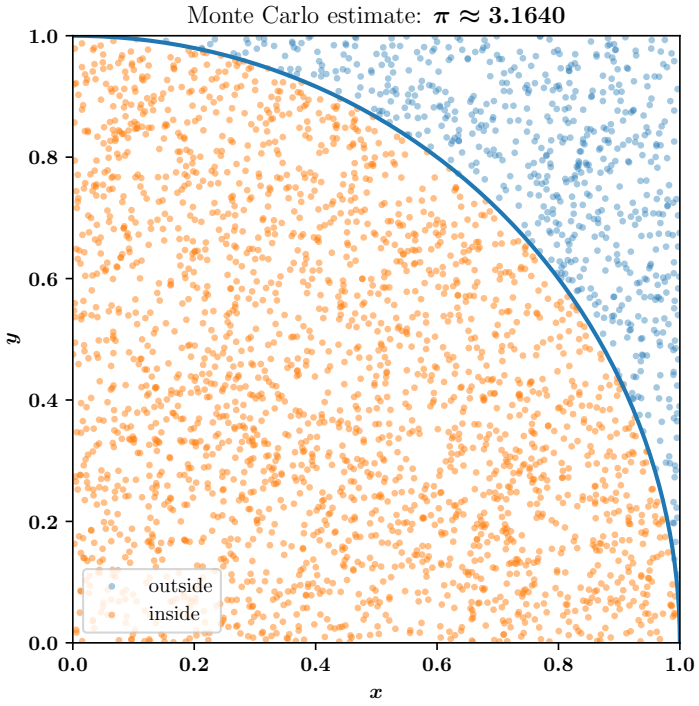

Figure 5.7.: Monte Carlo estimate of π using random points in a unit square. The fraction of points inside the quarter circle determines the estimate.

This convergence is slow in one dimension compared to high-order deterministic quadrature. The strength of Monte Carlo integration is that the convergence rate does not explicitly worsen with dimension. This makes Monte Carlo methods essential for high-dimensional problems.

5.7.1. Estimating π

A classic example is the estimation of π. Draw points uniformly in the unit square. The fraction of points inside the quarter circle $x^2 + y^2 \leq 1$ estimates the area ratio between the quarter circle and the square:

$$\frac{N_{\text{in}}}{N} \approx \frac{\pi/4}{1}. \tag{5.68}$$

Therefore,

$$\pi \approx 4\frac{N_{\text{in}}}{N}. \tag{5.69}$$

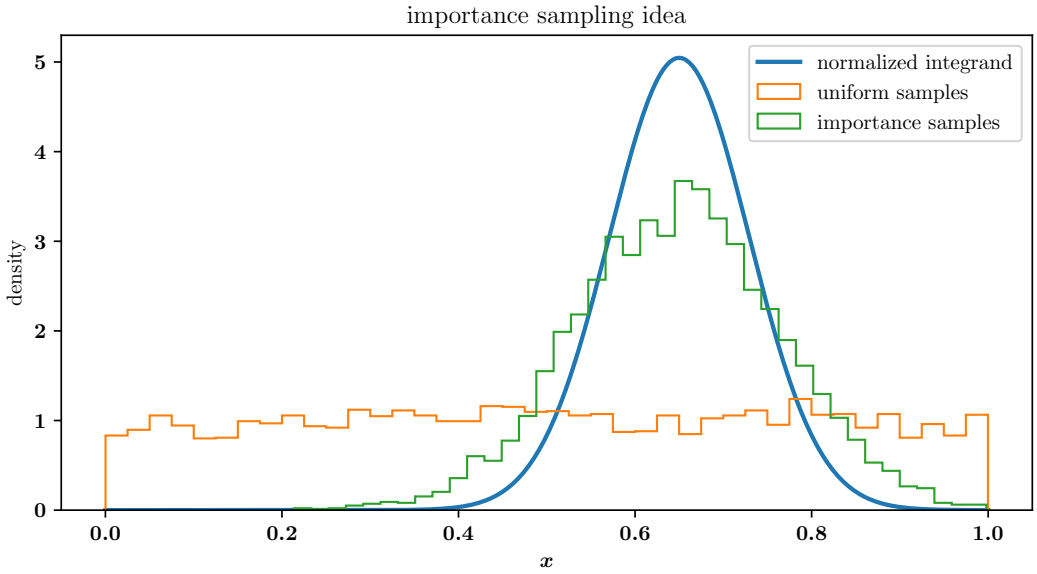

Figure 5.8.: Importance sampling for a sharply peaked function. Uniform samples waste many points in unimportant regions, whereas importance samples concentrate near the dominant contribution.

5.7.2. Importance sampling

Uniform sampling is not always efficient. If most of the contribution to an integral comes from a small part of the domain, it is better to sample more frequently where the integrand is large.

Let $g(x)$ be a normalized probability density with $g(x) > 0$ wherever $f(x)$ contributes. Then

$$I = \int f(x)\,dx = \int \frac{f(x)}{g(x)} g(x)\,dx. \tag{5.70}$$

If we draw samples x_i from $g(x)$ instead of uniformly, the estimator becomes

$$I_N = \frac{1}{N} \sum_{i=1}^{N} \frac{f(x_i)}{g(x_i)}. \tag{5.71}$$

A good choice of $g(x)$ can reduce the variance dramatically. A poor choice can make the variance worse. The art of importance sampling is to choose a proposal distribution that resembles the important parts of the integrand.

5.8. Random walks and diffusion

5.8.1. Simple linear diffusion

Random numbers also provide a direct microscopic model of diffusion. Consider a one-dimensional random walk with step size Δx. At every step the particle moves left or right with equal probability:

$$x_{n+1} = x_n \pm \Delta x. \tag{5.72}$$

After N steps, the expectation value of the position is

$$\langle x_N \rangle = 0, \tag{5.73}$$

but the mean squared displacement grows linearly:

$$\langle x_N^2 \rangle = N \Delta x^2. \tag{5.74}$$

To see why the mean squared displacement grows linearly with the number of steps, we write the position after N steps as a sum of individual random increments,

$$x_N = \sum_{i=1}^{N} \xi_i, \tag{5.75}$$

where each step ξ_i is either $+\Delta x$ or $-\Delta x$ with equal probability. Therefore,

$$\langle \xi_i \rangle = 0 \quad \text{and} \quad \langle \xi_i^2 \rangle = \Delta x^2. \tag{5.76}$$

The mean position is then

$$\langle x_N \rangle = \left\langle \sum_{i=1}^{N} \xi_i \right\rangle = \sum_{i=1}^{N} \langle \xi_i \rangle = 0. \tag{5.77}$$

For the mean squared displacement, we compute

$$\langle x_N^2 \rangle = \left\langle \left(\sum_{i=1}^{N} \xi_i \right)^2 \right\rangle. \tag{5.78}$$

Expanding the square gives

$$\langle x_N^2 \rangle = \left\langle \sum_{i=1}^{N} \xi_i^2 + 2 \sum_{i<j} \xi_i \xi_j \right\rangle. \tag{5.79}$$

The first term gives

$$\left\langle \sum_{i=1}^{N} \xi_i^2 \right\rangle = \sum_{i=1}^{N} \langle \xi_i^2 \rangle = N \Delta x^2. \tag{5.80}$$

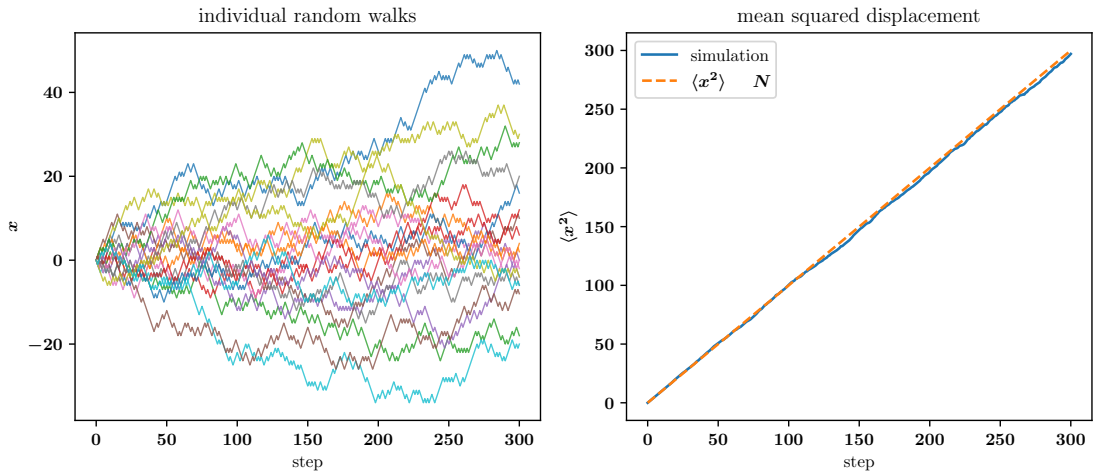

Figure 5.9.: One-dimensional random walks. Individual trajectories fluctuate strongly, but the ensemble mean squared displacement grows linearly in time.

The cross terms vanish because different steps are statistically independent and have zero mean:

$$\langle \xi_i \xi_j \rangle = \langle \xi_i \rangle \langle \xi_j \rangle = 0 \qquad \text{for } i \neq j. \tag{5.81}$$

Therefore,

$$\langle x_N^2 \rangle = N \Delta x^2. \tag{5.82}$$

The linear growth occurs because the variances of independent random steps add, whereas the positive and negative displacements cancel only in the mean.

If the time step is Δt, then $t = N\Delta t$ and

$$\langle x^2(t) \rangle = 2Dt, \tag{5.83}$$

where the diffusion coefficient is

$$D = \frac{\Delta x^2}{2\Delta t}. \tag{5.84}$$

This simple model connects random numbers to Brownian motion, heat conduction, particle diffusion, and cosmic-ray transport.

5.8.2. Ornstein–Uhlenbeck Process

The Ornstein–Uhlenbeck (OU) process is one of the most important stochastic processes in computational physics. It describes diffusion in the presence of a restoring force and is frequently used to model

- Brownian motion with drag,

- turbulent fluctuations,
- noisy relaxation processes.

In contrast to a pure random walk, the OU process does not diffuse freely to arbitrarily large values. Instead, the system fluctuates around an equilibrium state because a deterministic restoring force continuously pulls the variable back toward a preferred value.

The stochastic differential equation of the OU process is

$$dx = -\theta(x - \mu)\, dt + \sigma\, dW_t \tag{5.85}$$

where

- θ controls the relaxation strength,
- μ is the equilibrium position,
- σ controls the noise amplitude,
- dW_t is a Wiener process increment.

The equation consists of two competing parts:

- A deterministic drift term

  $$-\theta(x - \mu) \tag{5.86}$$

  which pulls the trajectory toward the equilibrium position.

- A stochastic diffusion term

  $$\sigma dW_t \tag{5.87}$$

  which continuously perturbs the system with random fluctuations.

The Wiener increment dW_t represents infinitesimal Brownian motion. Over a finite timestep Δt, the Wiener process has the properties

$$\langle \Delta W \rangle = 0 \quad \text{and} \quad \langle (\Delta W)^2 \rangle \propto \Delta t \tag{5.88}$$

which means that the stochastic fluctuations grow proportionally to $\sqrt{\Delta t}$.

Euler–Maruyama discretization

The Ornstein–Uhlenbeck equation is a stochastic differential equation (SDE). Just as ordinary differential equations can be solved numerically using Euler's method, stochastic differential equations can be discretized using the Euler–Maruyama scheme.

To understand this, recall the standard Euler method for an ordinary differential equation (see later in the lecture in more detail)

$$\frac{dx}{dt} = f(x) \tag{5.89}$$

which leads to the discretization

$$x_{n+1} = x_n + f(x_n)\Delta t \tag{5.90}$$

For stochastic equations, we additionally need to include the random Wiener increment. Since Wiener increments scale as

$$\Delta W \sim \sqrt{\Delta t} \tag{5.91}$$

the stochastic contribution becomes

$$\Delta W = \sqrt{\Delta t}\,\eta_n \tag{5.92}$$

where η_n are independent Gaussian random numbers with

$$\eta_n \sim \mathcal{N}(0,1) \tag{5.93}$$

Using this approximation, the OU process becomes

$$x_{n+1} = x_n - \theta(x_n - \mu)\Delta t + \sigma\sqrt{\Delta t}\,\eta_n \tag{5.94}$$

This equation is the Euler–Maruyama discretization of the Ornstein–Uhlenbeck process.

The update rule has an intuitive interpretation:

- The deterministic term $-\theta(x_n - \mu)\Delta t$ moves the trajectory toward equilibrium.
- The stochastic term $\sigma\sqrt{\Delta t}\,\eta_n$ adds Gaussian random kicks at every timestep.

The competition between relaxation and stochastic forcing produces fluctuating trajectories that remain statistically confined around the equilibrium value.

Physical interpretation

The OU process is closely related to Brownian motion with friction. For example, the velocity of a particle inside a fluid experiences:

- random collisions with surrounding molecules,
- viscous drag that slows the particle down.

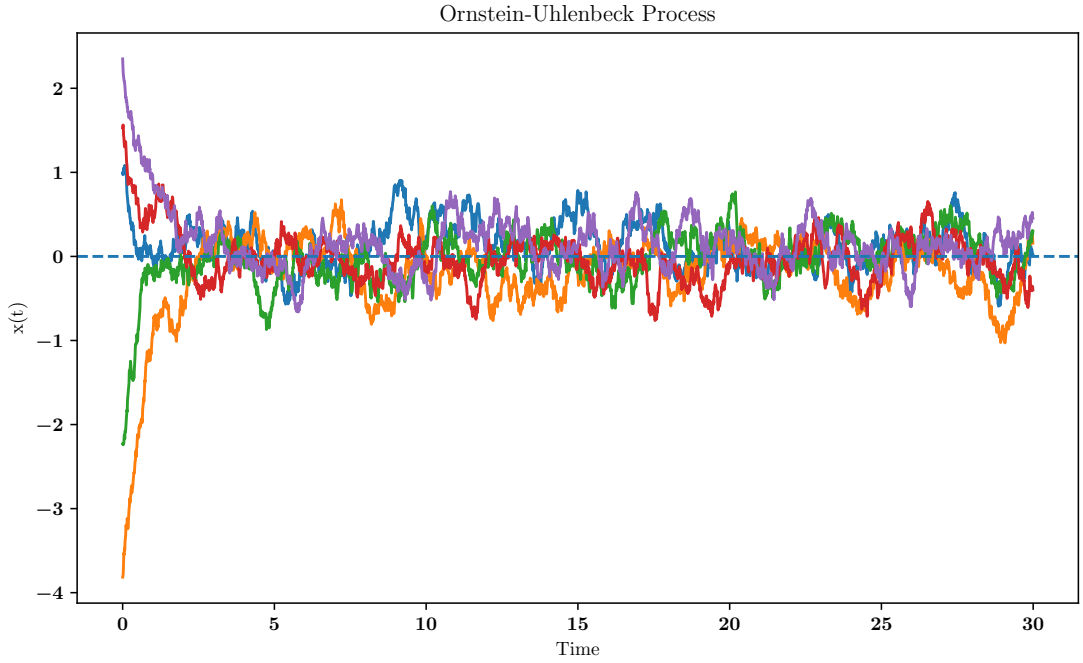

Figure 5.10.: Example realizations of the Ornstein–Uhlenbeck process. The restoring force prevents unlimited diffusion and keeps the trajectories close to the equilibrium position.

Without drag, the velocity would execute an unrestricted random walk. The restoring term prevents this and produces a stationary distribution. For sufficiently long times, the probability distribution approaches a Gaussian equilibrium distribution centered around μ.

An important characteristic timescale is

$$\tau = \frac{1}{\theta} \tag{5.95}$$

which describes how quickly the system relaxes back toward equilibrium.

Figure 5.10 shows several realizations of the Ornstein–Uhlenbeck process. In contrast to a free random walk, the trajectories remain statistically confined around the equilibrium value.

Application: stochastic turbulence driving

A particularly important application of the Ornstein–Uhlenbeck process in computational (astro-)physics is stochastic turbulence driving.

In many numerical simulations, turbulence cannot emerge naturally because the simulation domain is too small or because the physical driving processes are not modeled

explicitly. Instead, turbulent motions are injected artificially through a stochastic forcing field.

A simple white-noise driving force would change completely randomly from one timestep to the next. Such forcing is often unphysical because real turbulent flows exhibit finite temporal correlations. Large-scale eddies persist for a finite time before they decay.

The Ornstein–Uhlenbeck process provides a natural way to generate smoothly varying stochastic forcing with a controlled correlation time.

In turbulence driving, the stochastic variable is typically not a particle position but instead the amplitude of a driving mode in Fourier space. Each driving mode evolves according to an equation of the form

$$d\mathbf{f} = -\frac{\mathbf{f}}{\tau}\,dt + \sigma\,dW_t \tag{5.96}$$

where

- $\mathbf{f}$ is the turbulent forcing field,
- τ is the autocorrelation or driving timescale,
- σ controls the forcing strength.

The relaxation timescale τ determines how long a forcing pattern persists before it changes significantly. In turbulence simulations, this timescale is often chosen to be comparable to the turbulent crossing time,

$$t_{\rm cross} = \frac{L}{v_{\rm rms}} \tag{5.97}$$

where

- L is the turbulence driving scale,
- $v_{\rm rms}$ is the root-mean-square turbulent velocity.

This choice ensures that the forcing evolves on physically meaningful timescales similar to the turnover time of the largest turbulent eddies.

As a simple illustration you can compare this process to the stirring of your coffee with a spoon. The spoon should stir the coffee with a random overall character, so not just *rotating* the fluid inside the mug. The way the spoon trajectory changes should be comparable to the fluid motions: a fluctuating spoon in milliseconds will not stir the coffee as expected. Similarly, only changing the motion of the spoon over the time scale of a hour neither "properly" stirs the coffee.

Using the Euler–Maruyama discretization, the forcing update becomes

$$\mathbf{f}_{n+1} = \mathbf{f}_n - \frac{\mathbf{f}_n}{\tau}\Delta t + \sigma\sqrt{\Delta t}\,\boldsymbol{\eta}_n \tag{5.98}$$

where $\boldsymbol{\eta}_n$ is a Gaussian random vector.

Compared to uncorrelated white-noise forcing, Ornstein–Uhlenbeck driving produces smoother temporal evolution and avoids unrealistically abrupt changes in the forcing field. This method is widely used in simulations of (interstellar) turbulence, molecular clouds, cosmic ray transport simulations. The resulting turbulent flow develops a cascade of kinetic energy from large driving scales toward smaller dissipative scales, producing the characteristic multi-scale structure of turbulence.

5.9. Markov Chain Monte Carlo (MCMC)

5.9.1. Sampling from Complex Distributions

Many scientific problems require samples from a probability distribution that is known only up to a normalization constant. For example, in statistical physics the Boltzmann distribution is

$$p(x) = \frac{1}{Z} \exp[-\beta E(x)], \tag{5.99}$$

where the partition function is

$$Z = \int \exp[-\beta E(x)]\, dx. \tag{5.100}$$

The normalization Z is often impossible to compute directly. Similar problems occur if the dimension of x is large, such that brute force normalizations are infeasible.

The goal is to compute the expectation value

$$\langle f(x) \rangle = \int f(x)\, p(x)\, dx \tag{5.101}$$

via a sequence of random samples $x^{(1)}, x^{(2)}, \ldots, x^{(N)}$ whose distribution converges to $p(x)$. Then estimated expectation value is

$$\langle f(x) \rangle \approx \frac{1}{N} \sum_{i=1}^{N} f(x^{(i)}). \tag{5.102}$$

This is the idea behind MCMC.

5.9.2. Markov Chains

A *Markov chain* is a sequence of random variables

$$x^{(0)} \to x^{(1)} \to x^{(2)} \to \cdots \tag{5.103}$$

where the next state depends only on the current one,

$$P(x^{(n+1)} \mid x^{(n)}, x^{(n-1)}, \ldots) = P(x^{(n+1)} \mid x^{(n)}) \tag{5.104}$$

with the design so that the stationary distribution of the chain is $p(x)$, the distribution we want to sample from. If the chain is:

- **ergodic** (all states can eventually be reached),
- and satisfies **detailed balance** with respect to $p(x)$,

then it will converge to $p(x)$ after a sufficient number of steps.

5.9.3. Transition Probabilities and Detailed Balance

Let:

- $q(x \to x')$: the **proposal distribution**, i.e. the probability of proposing state x' from current state x,
- $a(x \to x')$: the **acceptance probability** for moving from x to x',
- $T(x \to x')$: the full transition probability of the Markov chain, given by

$$T(x \to x') = q(x \to x') \cdot a(x \to x') \tag{5.105}$$

The **detailed balance condition** states that

$$p(x)\, T(x \to x') = p(x')\, T(x' \to x) \tag{5.106}$$

i.e. the probability flow from x to x' is balanced by the reverse flow.

5.9.4. The Metropolis–Hastings Algorithm

The Metropolis–Hastings algorithm constructs a Markov chain that satisfies detailed balance with respect to $p(x)$.

Algorithm:

1. Start at some initial state $x^{(0)}$
2. For each step:
   a) Propose a new state $x' \sim q(x \to x')$
   b) Compute the acceptance probability:

$$a(x \to x') = \min\left(1, \frac{p(x')\, q(x' \to x)}{p(x)\, q(x \to x')}\right) \tag{5.107}$$

   c) Accept the move with probability $a(x \to x')$?
   draw $u \sim \mathcal{U}(0,1)$ and check $u \leq a$
      - if yes, set $x^{(n+1)} = x'$
      - if no, set $x^{(n+1)} = x^{(n)}$
3. Repeat for N steps

Special case: Symmetric proposals If $q(x \to x') = q(x' \to x)$ (e.g., Gaussian or uniform symmetric steps), the acceptance simplifies to:

$$a(x \to x') = \min\left(1, \frac{p(x')}{p(x)}\right) \tag{5.108}$$

5.9.5. Notes

- The samples are **correlated**: Each new sample depends on the previous one (via proposal and acceptance). As a result, the samples are statistically dependent, and nearby samples in the chain are typically autocorrelated. This means that averages converge more slowly.
- Burn-in (Thermalisation): At the beginning of the chain, the state may not represent the stationary distribution $p(x)$, especially if the initial guess is far from typical. This non-equilibrium phase is called the *burn-in period*. To avoid bias, discard the first N_{burn} samples:

  $$x^{(0)}, x^{(1)}, \ldots, x^{(N_{\text{burn}})} \quad \text{are ignored} \tag{5.109}$$

- Thinning: MCMC samples are correlated; to reduce this autocorrelation in saved data, one may keep only every kth sample:

  $$x^{(0)}, x^{(k)}, x^{(2k)}, \ldots \tag{5.110}$$

  This is called *thinning*. While it may reduce correlation in the output, it also discards potentially useful samples and does not improve statistical efficiency. Thinning is optional and usually only needed for storage reduction or for visualising less noisy chains.
- Alternative: Instead of thinning, one can use the full chain and estimate the *autocorrelation time* to compute correct error bars.
- If $p(x)$ is only known up to a constant (e.g. $p(x) \propto e^{-E(x)}$), the normalisation cancels in the acceptance ratio
- The efficiency of the algorithm depends on the proposal distribution $q(x \to x')$

5.10. The Ising Model

5.10.1. Motivation and Definition

The Ising model is a minimal lattice model for collective behaviour in magnetic materials, binary alloys, etc. Each lattice site i carries a *spin* variable

$$s_i = \pm 1, \tag{5.111}$$

representing a magnetic moment pointing "up" (+1) or "down" (−1). In its simplest (d-dimensional, nearest-neighbour) form the Hamiltonian is

$$\mathcal{H}(\{s\}) \;=\; -J\sum_{\langle i,j\rangle} s_i s_j \;-\; h\sum_i s_i. \tag{5.112}$$

- $J > 0$ ("ferromagnetic coupling") favours aligned neighbours.
- $h \equiv \mu B/k_{\mathrm{B}}T$ is the reduced external magnetic field.
- $\langle i, j\rangle$ denotes pairs of nearest neighbours on a cubic, square, triangular etc. lattice.

Boltzmann Distribution At temperature T the probability of configuration $\{s\}$ is

$$p(\{s\}) \;=\; \frac{\exp\bigl(-\beta\mathcal{H}(\{s\})\bigr)}{Z(\beta)}, \qquad \beta = \frac{1}{k_{\mathrm{B}}T}, \tag{5.113}$$

with partition function $Z(\beta) = \sum_{\{s\}} e^{-\beta\mathcal{H}}$. The normalising sum scales as 2^N, thus becomes intractable for $N \gtrsim 10^4$.

Key Observables

1. Magnetisation:

$$M(\{s\}) = \sum_i s_i \tag{5.114}$$

$$\langle M\rangle = \sum_{\{s\}} M(\{s\})p(\{s\}) \tag{5.115}$$

$$\tag{5.116}$$

2. Energy:

$$E(\{s\}) = \mathcal{H}(\{s\}) \tag{5.117}$$

$$\langle E\rangle = \sum_{\{s\}} E(\{s\})p(\{s\}) \tag{5.118}$$

   The magnetisation $M(\{s\})$ (energy $E(\{s\})$) is simply the sum over all lattice points. The mean magnetisation (mean energy) includes the sum over *all* possible states with a nested simple sum over all lattice points, so $\langle M\rangle$ becomes intractible to compute brute force.

3. Response functions:

$$C_V = \frac{\partial\langle E\rangle}{\partial T} = \frac{1}{k_B T^2}\left(\langle E^2\rangle - \langle E\rangle^2\right) = k_B\beta^2\left(\langle E^2\rangle - \langle E\rangle^2\right) \tag{5.119}$$

$$\chi = \frac{\partial\langle M\rangle}{\partial h} = \frac{1}{k_B T}\left(\langle M^2\rangle - \langle M\rangle^2\right) = \beta\left(\langle M^2\rangle - \langle M\rangle^2\right) \tag{5.120}$$

The response functions reflect how the mean system energy (magnetisation) changes with changes in the temperature (magnetic field). A direct computation of the approximate quantity $C_V \approx \frac{\Delta\langle E\rangle}{\Delta T}$ requires running the system for multiple temperatures with linearly increasing numerical cost per value of T.

We can rewrite the derivatives as fluctuations in the energy and the magnetisation, (see Appendix B.1 for a derivation), which simplifies the computation significantly, if we can compute the fluctuations easily.

Physics Notes:

- Close to the critical temperature T_c (for $d \geq 2$) these quantities exhibit power-law divergences $C_V \sim |t|^{-\alpha}$, $\chi \sim |t|^{-\gamma}$, where $t = (T - T_c)/T_c$.
- The dimensionality of the system matters
  - $d = 1$: no phase transition at $T > 0$ (exact solution).
  - $d = 2$: Onsager's exact $T_c = 2J/[\ln(1 + \sqrt{2})]k_\mathrm{B}$.
  - $d \geq 3$: finite T_c; critical exponents from Renormalization Group or numerics.

Boundary Conditions Periodic (toroidal) boundaries minimise surface artefacts: $(i + 1) \bmod L$ couples the last spin back to the first.

5.10.2. MCMC

Enumerating 2^N states is hopeless; we instead draw a *Markov chain* of configurations

$$\{s\}^{(0)} \;\to\; \{s\}^{(1)} \;\to\; \ldots \;\to\; \{s\}^{(n)} \tag{5.121}$$

whose stationary distribution is $p(\{s\})$. Once the chain has "thermalised" (burn-in), time averages converge to ensemble averages (ergodic theorem).

Detailed Balance A sufficient condition:

$$p(\{s\})\, T\big(\{s\} \to \{s'\}\big) \;=\; p(\{s'\})\, T\big(\{s'\} \to \{s\}\big), \tag{5.122}$$

where T is the transition probability (do not confuse with the temperature T here).

Metropolis Algorithm

1. **Pick a random site** i (uniformly among N sites).
2. **Propose** to flip its spin: $s_i \mapsto -s_i$. All others unchanged.
   $q(x \to x') : s_i \to -s_i$
   $q(x \to x') = 1$

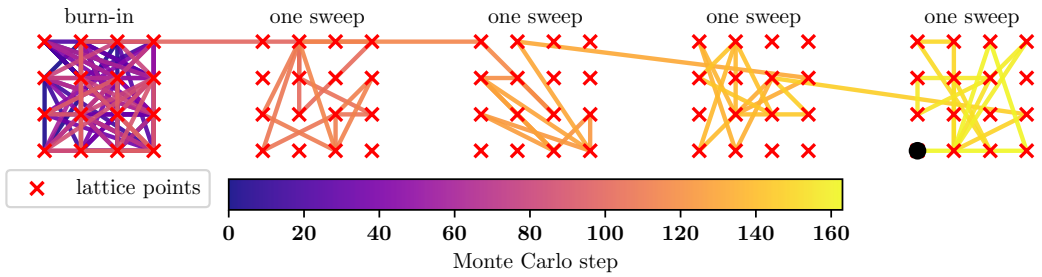

Figure 5.11.: Illustration of the MCMC simulation in the Ising model. Shown are the burn-in phase and four sweeps on a 4×4 lattice.

3. **Compute energy change**

$$\Delta E = 2Js_i \sum_{j \in \langle i \rangle} s_j \; + \; 2hs_i. \tag{5.123}$$

Only nearest neighbours matter $\Rightarrow \mathcal{O}(1)$ cost.

4. **Acceptance probability**

$$p(\{s\}) = \frac{\exp\bigl(-\beta \mathcal{H}(\{s\})\bigr)}{Z(\beta)} \tag{5.124}$$

$$p(\{s'\}) = \frac{\exp\bigl(-\beta \mathcal{H}(\{s'\})\bigr)}{Z(\beta)}, \tag{5.125}$$

$$a(\{s\} \to \{s'\}) = \min\left(1, \; \frac{p(\{s'\})\, q(\{s'\} \to \{s\})}{p(\{s\})\, q(\{s\} \to \{s'\})}\right) \tag{5.126}$$

$$= \min\left(1, \; \frac{p(\{s'\})}{p(\{s\})}\right) \tag{5.127}$$

$$= \min\left(1, \; e^{-\beta \Delta E}\right). \tag{5.128}$$

Implement by drawing $u \sim \mathcal{U}(0,1)$ and accepting if $u \leq p_{\text{acc}}$.

5. **Loop** $\mathcal{O}(N)$ times $\Rightarrow$ one *Monte-Carlo sweep*. After a few thousand sweeps (temperature-dependent) the chain is typically equilibrated.

Monte Carlo sweep. We define one *sweep* as N spin-flip attempts, i.e. one per site on average. Each sweep costs $\mathcal{O}(N)$ operations. To reach equilibrium (burn-in), we typically require 100–1000 sweeps, depending on system size and temperature. After thermalisation, we perform N_{sample} additional sweeps to record observables, see illustration for a small 4×4 lattice in Fig. 5.11. Therefore, the total cost scales as:

$$\text{Total work} \; \sim \; (\text{burn-in} + \text{samples}) \times N \; = \; \mathcal{O}(N_{\text{sweeps}} \cdot N) \tag{5.129}$$

For a lattice of $L \times L$ spins, this is $\mathcal{O}(L^2 N_{\text{sweeps}})$. Because only local information is needed for a spin iteration, a sweep costs $\mathcal{O}(L^2 = N)$, allowing lattices of $N \sim 10^7$ on a laptop.

Direct computation. The partition function reads

$$Z = \sum_{\text{configs}} e^{-\beta E[\text{config}]} \tag{5.130}$$

with exponentially growing number of configurations

$$\text{Total configs} = 2^N \quad \Rightarrow \quad \text{Cost} = \mathcal{O}(N \cdot 2^N) \tag{5.131}$$

Autocorrelation and Effective Sample Size

In Markov Chain Monte Carlo (MCMC) simulations, subsequent configurations are generated sequentially from previous configurations. Therefore, measurements taken along the Markov chain are generally not statistically independent. This affects the estimation of statistical uncertainties.

Suppose we measure an observable O along the Markov chain:

$$O_1, O_2, O_3, \ldots \tag{5.132}$$

For completely independent measurements, the variance of the sample mean

$$\bar{O} = \frac{1}{N} \sum_{i=1}^{N} O_i \tag{5.133}$$

would scale as

$$\text{Var}(\bar{O}) = \frac{\text{Var}(O)}{N}. \tag{5.134}$$

However, in MCMC simulations neighboring samples are correlated because each configuration differs only slightly from the previous one. In the Ising model, for example, a single-spin flip changes the magnetization and energy only locally, such that consecutive measurements retain memory of previous states.

To quantify this memory effect, one defines the autocorrelation function

$$C_O(t) = \langle O_{n+t} O_n \rangle - \langle O \rangle^2, \tag{5.135}$$

where t is the lag in Monte Carlo time. At zero lag,

$$C_O(0) = \langle O^2 \rangle - \langle O \rangle^2 = \text{Var}(O). \tag{5.136}$$

It is useful to normalize the autocorrelation function:

$$\rho_O(t) = \frac{C_O(t)}{C_O(0)}. \tag{5.137}$$

The normalized autocorrelation function satisfies

$$\rho_O(0) = 1, \tag{5.138}$$

and typically decays approximately exponentially:

$$\rho_O(t) \sim e^{-t/\tau}. \tag{5.139}$$

The integrated autocorrelation time is then defined as

$$\tau_{\text{int}} = \frac{1}{2} + \sum_{t=1}^{\infty} \rho_O(t) = \frac{1}{2} + \sum_{t=1}^{\infty} \frac{C_O(t)}{C_O(0)}. \tag{5.140}$$

This quantity measures how long the Markov chain remembers its previous configurations[1]. Large values of τ_{int} indicate slow decorrelation. The variance of the sample mean becomes

$$\text{Var}(\bar{O}) = \frac{2\tau_{\text{int}}}{N}\text{Var}(O), \tag{5.144}$$

instead of the independent result

$$\text{Var}(\bar{O}) = \frac{\text{Var}(O)}{N}. \tag{5.145}$$

This motivates the definition of the effective number of statistically independent samples:

$$N_{\text{eff}} \approx \frac{N}{2\tau_{\text{int}}}. \tag{5.146}$$

Thus, although the simulation may contain N measurements, correlations reduce the statistical information to approximately N_{eff} independent samples.

Near the critical temperature of the Ising model, autocorrelation times become very large because large spin domains evolve only slowly. This phenomenon is known as **critical slowing down**. As a result, simulations near phase transitions require substantially more Monte Carlo steps to obtain statistically independent measurements.

[1] The factor $\frac{1}{2}$ in

$$\tau_{\text{int}} = \frac{1}{2} + \sum_{t=1}^{\infty} \rho(t) \tag{5.141}$$

is a normalization convention that arises naturally in the variance of the sample mean:

$$\text{Var}(\bar{O}) = \frac{\text{Var}(O)}{N}\left[1 + 2\sum_{t=1}^{\infty} \rho(t)\right]. \tag{5.142}$$

Defining τ_{int} in this way gives the compact result

$$\text{Var}(\bar{O}) = \frac{2\tau_{\text{int}}}{N}\text{Var}(O). \tag{5.143}$$

For completely independent samples, $\rho(t > 0) = 0$, such that $\tau_{\text{int}} = \frac{1}{2}$ and the usual result $\text{Var}(\bar{O}) = \text{Var}(O)/N$ is recovered.